Ain Shams Engineering Journal 14 (2023) 101828

Contents lists available at ScienceDirect

Ain Shams Engineering Journal

journal homepage: www.sciencedirect.com

Deep-learning cell-delay modeling for static timing analysis

Waseem Raslan [a,*], Yehea Ismail [b]

[a] IC Verification Solutions, Siemens EDA, Siemens Digital Industries Software, Cairo, Egypt
[b] Center of Nanoelectronics and Devices, CND, The American University in Cairo, Cairo, Egypt

ARTICLE INFO

Article history:
Received 18 November 2021
Revised 24 April 2022
Accepted 9 May 2022
Available online 28 May 2022

Keywords:
Electronic Design Automation
Machine Learning
Deep Learning
Autoencoders
Cell Characterization
Effective Current Source Model

ABSTRACT

Delay and transition timetables plus voltage waveforms are used to characterize standard cell delays. More accurate models explode cell library size and degrades design flow performance. Our proposed deep learning non-linear delay model, DL-NLDM, technique outperformed 7×7 NLDM-LUT in average percentage errors with up to 1.4% error compared to SPICE and outperformed the non-standard 100×100 NLDM-LUT in maximum percentage errors. The proposed DL Autoencoder-based waveform compression outperformed singular value decomposition by 1.79×. Additionally, a novel DL waveform-delay model, DL-WFDM, models cell delays using encoded waveforms instead of delay and transition time. DL-WFDM outperformed DL-NLDM in maximum delay percentage errors.
© 2022 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Ain Shams University
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. Introduction

Cell delay is the time taken for the cell output to switch logic from the time at which cell input switched logic in a way that causes the output to change. Cell delay is a function of the input transition time and the effective load at the output pin. The effective output load is a combination of the net delays and the input capacitance at the subsequent cell input driven by this net. Therefore, the waveform delay and transition (slew) time are the two waveform parameters widely used in the non-linear delay model, NLDM, used to characterize logic gates to perform timing closure as explained in [8] and [1]. As the transistor feature size decreases, NLDM fails short to model the cell delay correctly. Therefore, Cadence introduced the Effective Current Source Model, ECSM, and got adopted as a Silicon Integration Initiative standard [11], and Synopsys introduced the Composite Current Source Model, CCS, [13]. Both ECSM and CCS depend on storing waveform information the library technology file to be able to model cell delays of more complex nature.

In NLDM, Delay and transition time lookup tables are created during the cell characterization process to model the cell delay and output transition time. Delay and transition timetables specify the corresponding timing value for few selected values of input waveform transition time and effective capacitance. Interpolation and extrapolation techniques are used to extract the timing values for input transition and capacitive load values that do not exist in the tables. On the other hand, deep-learning techniques [4] can generalize based on its trained dataset. Trained feed-forward neural network can better predict values beyond its trained dataset more accurately. Voltage and current waveforms are also used to obtain effective current source model and composite current source models respectively. Waveform compression techniques are needed to increase the waveform fidelity without increasing the technology file size. General data compressors can be used like gzip, zlib, and bz2 techniques [3]. Ramalingam et al. [2] proposed fixed waveform-basis principal component analysis. PCA, using the singular value decomposition, SVD, technique. Hatimi et al. [9] used an adaptive waveform-basis SVD to get better compression results of certain waveforms. They achieved compression

Abbreviations: DL, Deep Learning; NLDM-LUT, Nonlinear Delay Model Lookup Tables; DL-NLDM, Deep Learning Nonlinear Delay Model; ECSM, Effective Current Source Model; DL-WFDM, Deep Learning Waveform Delay Model; CCS, Composite Current Source; SVD, Singular Value Decomposition.

* Corresponding author at: Siemens Digital Industries Software, Cairo Festival Southern District, Plot 14B02, Building A3, Floor 5, Private Free Zone, New Cairo 11835, Egypt.

E-mail addresses: waseem_raslan@mentor.com (W. Raslan), y.ismail@aucegypt.edu (Y. Ismail).

Peer review under responsibility of Ain Shams University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.asej.2022.101828
2090-4479/© 2022 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Ain Shams University
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

ratios between 20 and 4 depending on the accuracy level. Recursive polynomial representation is used by Saurabh and Mittal [10]. Li and Yu proposed signal-value compression scheme achieving average of 26 compression ratio [5]. Abrishami et al. [6] proposed a neural network simulation framework to eliminate current source model lookup tables without performing waveform data compression to reduce memory latency and computation of current source model. Training Autoencoders requires a large dataset and intensive computation power. However, since the training phase is done offline, this intensive computation power can be tolerated for better performance. Fournier and Aloise [14] compared Autoencoders performance with PCA technique using different image datasets. They showed that PCA technique produces comparable accuracy to Autoencoders results at two orders of magnitude faster performance.

Several deep learning techniques to model cell delays are proposed in this research paper. First, a deep learning non-linear delay model, DL-NLDM, is proposed to replace the traditional non-linear delay model lookup tables, NLDM LUT, to produce higher degrees of accuracy over a wide range of input parameters. These DL-NLDM models are trained using measured cell and transition time delay at different input transition time and capacitive load values. The deep learning model ability to generalize allowed achieving better results than lookup tables. Next, an Autoencoder-based technique is proposed to compress ECSM waveforms that are used in advanced cell delay modeling techniques. This proposed Autoencoding technique outperformed singular value decomposition, SVD, technique in compression ratio and accuracy, and outperformed gzip and bz2 techniques in compression ratio. The same technique is used in compressing CCS waveforms; however, CCS is not the focus of this research paper. Finally, deep learning waveform delay model, DL-WFDM, is proposed instead of NLDM LUT or DL-NLDM models. In DL-WFDM, auto-encoded input/output waveforms parameters are proposed to model cell delays instead of the cell delay and the transition time used in the NLDM. SPICE-simulated cell input and output waveforms are used to train DL waveform Autoencoders. These trained Autoencoders can encode and decode the whole waveform into few parameters. The input/output encoded waveform parameters are then used together with the associated effective capacitance to train another DL model and generalize the cell behavior. The trained DL-WFDM model can then be utilized to produce the expected cell output encoded waveform given an encoded input. At any point of time, the cell output can be decoded back to its time-domain form using the corresponding waveform decoder produced earlier during the training phase. Raslan et. Yehia [15] proposed using DL Autoencoders to compress ECSM and CCS waveforms to reproduce waveforms with better fidelity with smaller cell library files. This research is extending this work it differs in the following aspects:

- Focusing on DL applications to cell delay modeling including waveform data compression.
- Introducing DL-NLDM and comparing its results with the traditional LUT model.
- Building on Autoencoder compression technique to propose DL-WFDM cell delay modeling.
- Proposing varying time 50, 100, and 150 points sampling and waveform compression technique.
- Comparing Autoencoder compression technique with general lossless compression techniques.
- Taking maximum waveform errors into consideration in waveform compression comparison.
- Changing the activation function of the Autoencoder middle layer to sigmoid to avoid data normalization when used in the proposed DL-WFDM.

The main contributions of this research paper are:

- Modeling cell delays using DL neural networks. This approach reduced the two tables characterizing the cell delay and transition time to one DL model. The DL-NLDM outperformed average percentage errors of the standard 7×7 LUT size and outperformed the non-standard 100×100 LUT in maximum percentage errors over the selected wide range of input values, and less than 1.4% average delay percentage error compared with SPICE simulation.
- Compressing ECSM voltage time waveforms to increase the waveform fidelity while reducing the technology file size using DL Autoencoders technique. This DL compression technique outperformed the SVD technique by 1.67× for 1000-points sampled waveforms, and by 1.79× for 150-points sampled waveforms. Autoencoder technique outperformed gzip and bz2 techniques compression by 45×.
- Modeling cells using DL-WFDM model trained on two-parameters encoded input waveforms plus capacitive load values as inputs and two-parameters encoded output waveforms. This proposed model produced better maximum delay percentage error over all NLDM LUT tables used in the experiments in addition to the proposed DL-NLDM.

2. Cell-delay modeling background

Cell characterization is the process of extracting important cell characteristics to build an accurate and efficient cell model that represents its behavior. The approach of trading accuracy with runtime performance is usually taken to guarantee adequate cell model that executes fast in different design flows. This paper focuses on voltage-based cell driver models; NLDM model and ECSM waveforms.

2.1. NLDM cell driver model

The most notable cell driver model is the NLDM as explained by Sharma [8]. The characterization process starts with connecting the cell output pin to an effective load capacitance and applying a ramp signal to the cell input. The ramp slew rate and the effective load capacitance are swept through defined a range to measure the output slew rate and transition time. Lookup tables of output slew rate and transition time parameters at in given input slew rate and the effective load capacitance are stored in technology files. Usually, two tables exist, one for the rising-edge and the second for the falling-edge. Static timing analysis tools make use of these technology tables to perform its timing analysis. NLDM ignores the maximum and minimum waveform values and all other waveform fluctuations as if focuses on transition time and delay. As the transistor feature size decreases, and the complexity of designs increases, timing analysis becomes more sensitive to signals over-shoot, under-shoot, and multiple crossings. Hence, more complex models, like ECSM, were introduced to increase cell model accuracy.

2.2. ECSM cell driver model

In addition to slew and transition-time lookup tables, the output voltage waveform at a given slew and output load is also stored in the library technology file. Waveform information is stored in the output voltage table ECSM format. The more samples taken to capture waveform information, the more accurate reconstructed waveform. There are several ways for sampling waveforms. This research focuses on sampling voltage at certain uniform time increments which results in the N-points voltage–time samples. Sampling N-points voltage–time samples can capture more impor-

tant waveform characteristics. Sampling N-points at variable time range at which the signal reaches 95%VDD or 5%VDD for rising and falling edge waveforms, respectively, can also capture important waveform characteristics. However, the accuracy of the reconstructed waveforms comes on the expense of the size of technology file on disk and the performance of static analysis tools making use of these files. Hence, trade-offs between voltage waveform accuracy and data size on disk must be applied.

3. Deep learning Autoencoders background

Feed-forward neural network is used to encode large data into a smaller set of parameters representing the original data. The core idea of Autoencoder is to train a deep feed-forward neural network to produce an output that matches the input data to a certain degree of error [4]. Zhang et al., [12], show several new applications to Autoencoders in convolutional neural networks, natural language processing, extreme learning machines and others. Wang et al., [16], concluded that Autoencoders can detect repetitive structure in addition to performing dimensionality reduction of trained data. The output layer must have the same size of the input layer. If the Autoencoder middle layer size is smaller than the number of inputs as shown in Fig. 1, then the Autoencoder can perform dimensionality reduction of data which the main usage of Autoencoders in this research. After training, the autoencoder is split at the middle layer into two models: an encoder model and a decoder model. The layers from the input layer to the middle layer represent the encoder model. When the original data is presented as input to the encoder model, the output values of the middle layer neurons are the encoded, compressed data of the original data. Therefore, the number of neurons in the middle layer is considered the number of encoding parameters. The layers from the middle layer to the output layer represent the decoder model with the middle layer playing as the input layer to the decoder model. The compressed encoded data – the output of the middle layer, are presented as input to the first decoder model layer to be decoded to reproduce the original data within certain degree of error.

Encoding large data points into a smaller number of parameters can be used in performing the required waveform data compression. The decoder model must be provided with the compressed data to be able to decompress and restore the original data.

4. Deep learning cell delay modeling

Deep learning models are proposed to replace NLDM look-up tables, DL-NLDM, and to encode ECSM voltage–time waveforms into very small number of parameters. Those parameters can be decoded again to the original waveform within certain error. The less parameters used to represent a complex waveform the better in terms of reduced size on disk and higher execution performance. The steps followed are:

- Generating NLDM data and ECSM waveforms.
- Preprocessing NLDM and ECSM waveform data.
- Designing deep-learning neural network models
- Training of deep-learning neural network models.

4.1. Generation and pre-processing of training data

4.1.1. Generating NLDM data and ECSM waveforms

MOS transistor model level 53 is used to perform transistor level simulation of NOR, NAND and INV cells. Fig. 2 shows the cell characterization setup; each cell is connected to a capacitive load and 1.2v VDD source. The slew rate of a ramp input is varied from 10 ps to 2 ns, whereas the capacitive load is varied from 0.1fF to 100fF.

In each simulation run, the 1000-points voltage–time samples are measured at the output pin to store the waveforms needed for ECSM models. Uniform time 1000 sampled points per waveform is selected to capture more complex waveform properties like overshoots, undershoots, rapid transitions and multiple crossings. Another sampling technique is devised to sample the most significant N samples before crossing 95%VDD or 5%VDD for rising or falling time waveforms respectively as shown in Fig. 3. The number of samples N is varied between 50, 100 and 150 points. Sampling 150-points gave better results than 50 and 100, therefore sampling results of 150-points are reported and compared with sampling uniform time 1000-points in this paper.

Total of 71,400 rising-edge and another 71,400 falling-edge voltage waveforms were captured for ECSM models. During the same simulation run, delay and transition time values are recorded against the input waveform transition time and effective load capacitance to train the NLDM deep learning models.

4.1.2. Preprocessing NLDM data and ECSM waveforms

Data normalization is a key step to perform before training deep learning networks. Preprocessing voltage–time ECSM waveform data needs to account for the over and under shoots. Assuming 50% possible overshoot or undershoot, then equation (1) normalizes waveform data between 0 and 1 including possible over or under-shoots.

$$Normalized\,WF = (WF + 0.5VDD)/(2 * VDD) \quad (1)$$

Pre-processing NLDM data is as simple as normalizing NLDM data values between 0 and 1 using equation (1).

$$Normalized\,NLDM\,data = (NLDM - Min.Value)/(Max.Value) \quad (2)$$

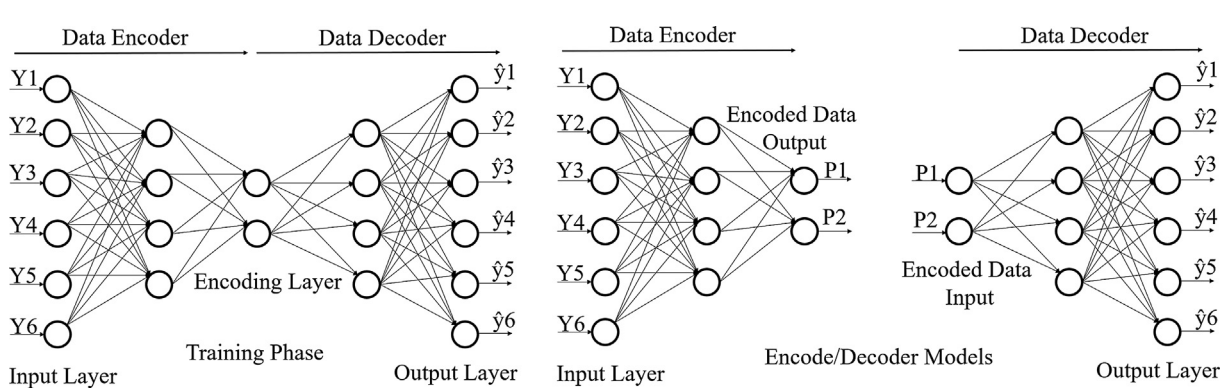

Fig. 1. Training phase and Encode/Decoder Models of DL Autoencoder.

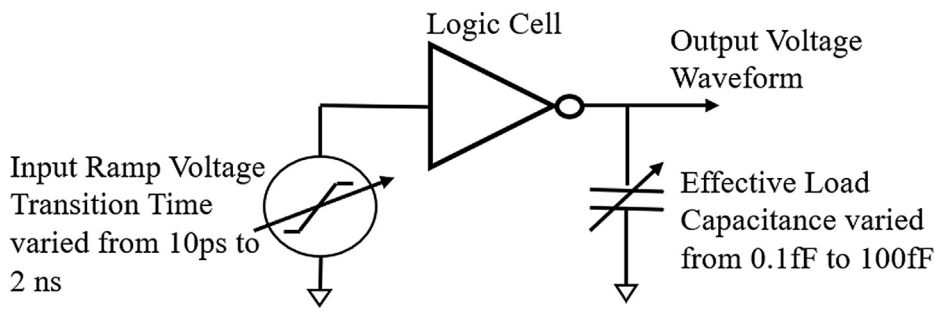

Fig. 2. Cell Characterization Transistor Level Simulation Setup.

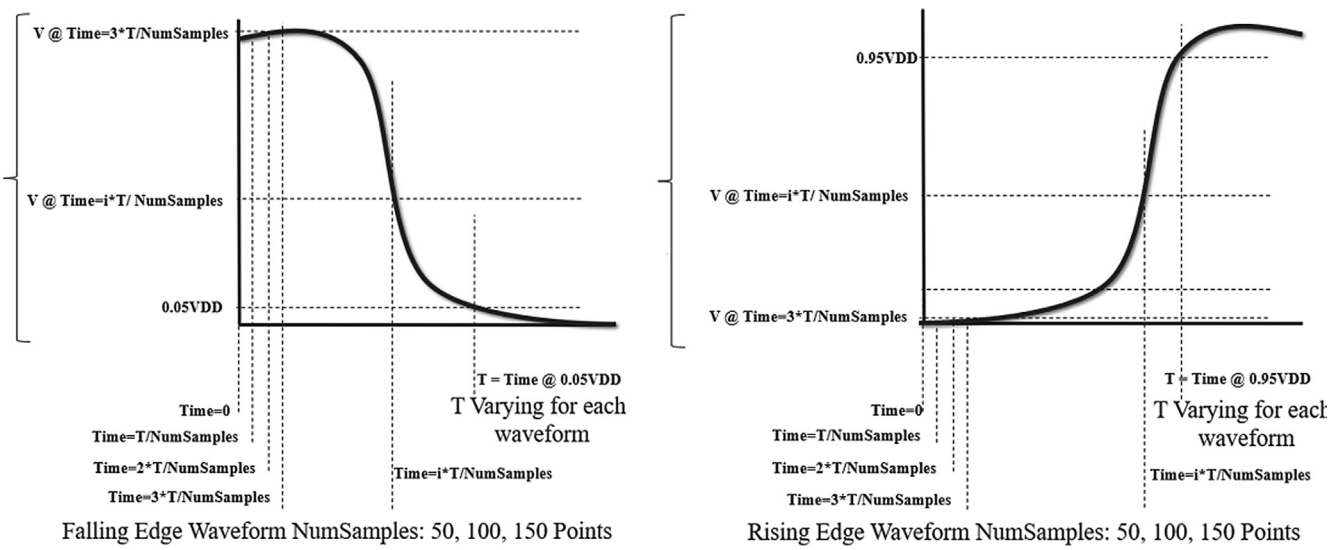

Fig. 3. Varying Time, 50, 100, 150 Sampling Points.

4.2. Deep learning NLDM cell delay modeling

The same cell characterization setup used to obtain the ECSM voltage waveforms are used to get the training data for NLDM models. For each waveform output, the cell delay is measured as well as the slew rate or the time needed to transition from 80% from/to 20% of VDD voltage. The training data is constructed using the input waveform transition time (slew rate) and effective load capacitance, and the output cell delay and the waveform slew rate. Feed-forward fully connected, FC, neural network is constructed and trained using this training data. Fifteen different deep learning model topologies are explored, one DL model and one training process is required for each of NAND, INV, and NOR. Table 1 shows the models that gave best modeling results.

The error rates of the deep-learning models are compared with the error rates of NLDM lookup tables, LUT, of different sizes. Though the de-facto size of NLDM LUT is 5x5 or 7×7 tables, DL-NLDM results are compared with other larger non-standard LUT sizes as shown in Tables 2, 3 and 4 for different cell types. To increase the accuracy of the LUT tables, two sets of tables are used for each cell/edge type; one LUT table for input ramp range of 10 ps to 1 ns and capacitive load range of 0.1fF to 1fF, and another LUT table for input ramp range of 1 ns to 2 ns and capacitive load range of 1fF to 100fF. On the other hand, a single DL-NLDM is trained on the whole input range. Offline training time is 20 s, retrieving time is almost the same for different cell types, so it's reported for NOR type only in Table 2.

The above tables show that the single DL-NLDM outperforms the rising and falling edge 7×7 dual-table LUT average error accuracy and outperforms 100×100 dual-table LUT in maximum percentage errors for both delay and transition time over the selected wide range of transition time and effective capacitance values. Table 2 shows also that the retrieving time of DL-NLDM is better than those of different NLDM-LUT sizes. The increased accuracy and better retrieving time come at the expense of offline DL-NLDM training time and the increased size of disk, however the DL-NLDM disk size is better than 100×100 LUT size.

Table 1
Deep Learning Model Structure.

Model ID	FC Neural Network Structure	Activation Functions
2	I,50,25,2	Relu for all layers, Sigmoid for output layer
3	I,100,2	I = 2 or 3
4	I,100,50,2	
6	I,200,100,25,2	
8	I,150,75,32,16,2	

4.3. ECSM voltage waveform deep learning Autoencoder-based compression

4.3.1. Designing Autoencoder models

Several Autoencoder models were designed and trained using the preprocessed ECSM waveform data. The model topologies that resulted in best training performance are listed in Table 5. Each model has an input layer I, and output layer O of the same size that

Table 2
DL-NLDM best trained NOR models Mean/Max Percentage Error rates versus different LUT sizes.

	NOR Falling Edge				NOR Rising Edge			
Delay Model	**LUT** 7×7	**LUT 25x25**	**LUT** 100×100	**DL-NLDM, Model ID = 6**	**LUT** 7×7	**LUT 25x25**	**LUT** 100×100	**DL-NLDM, Model ID = 6**
% error	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max
Delay Time	0.53/15.46	0.22/15.46	0.16/18.1	0.48/3.56	0.91/16.54	0.34/16.54	0.19/19.0	0.71/5.95
Tr Time	1.65/33.49	0.89/33.49	0.51/52.8	1.19/8.00	4.24/43.85	2.12/48.06	0.91/89.2	3.20/36.33
MSE	–	–	–	1.76E-05	–	–	–	6.29E-05
Disk size-KB	2	25.2	408	109	2	25.2	408	109
Retrieving time (s)	0.18 s	0.26 s	1.28 s	0.106 s	0.18 s	0.26 s	1.28 s	0.106 s

Table 3
DL-NLDM best trained INV models Mean/Max Percentage Error rates versus different LUT sizes.

	INV Falling Edge				INV Rising Edge			
Delay Model	**LUT** 7×7	**LUT 25x25**	**LUT** 100×100	**DL-NLDM, Model ID = 6**	**LUT** 7×7	**LUT 25x25**	**LUT** 100×100	**DL-NLDM, Model ID = 6**
% error	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max
Delay Time	0.70/26.31	0.37/26.31	0.29/33.1	0.60/10.41	2.82/51.53	0.97/31.51	0.40/43.4	1.41/42.25
Tr Time	2.67/39.85	1.42/41.58	0.62/59.8	1.91/22.67	9.25/128.6	3.94/153.7	1.0/142.5	4.86/51.80
MSE	–	–	–	3.48E-05	–	–	–	2.35E-04
Disk size-KB	2	25.2	408	109	2	25.2	408	109

Table 4
DL-NLDM best trained NAND models Mean/Max Percentage Error rates versus different LUT sizes.

	NAND Falling Edge				NAND Rising Edge			
Delay Model	**LUT** 7×7	**LUT 25x25**	**LUT** 100×100	**DL-NLDM, Model ID = 6**	**LUT** 7×7	**LUT 25x25**	**LUT** 100×100	**DL-NLDM, Model ID = 6**
% error	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max	Avg/Max
Delay Time	0.61/23.44	0.26/23.44	0.24/28.6	0.49/7.84	1.88/27.38	0.68/22.97	0.29/27.9	0.79/12.30
Tr Time	1.32/38.10	0.65/38.10	0.46/50.8	0.85/7.51	5.54/57.95	2.29/110.5	0.78/54.5	2.69/74.66
MSE	–	–	–	2.91E-05	–	–	–	8.66E-05
Disk size-KB	2	25.2	408	109	2	25.2	408	79

Table 5
Autoencoder Models for Parameter P = 16,8,4,2,1.

ID	Autoencoder Models	
	Model Topology	Activation Function
17	I,128,64,32, P,32,64,128,O	Relu for all layers, except Sigmoid for the middle and last layer
15	I,64,32, P,32,64,O	I = O = 1000 or 150
2	I,1024,16, P,16,1024,O	

is equal to the number of waveform sample points. For each model, the number of the encoded parameters P is varied between 8, 4, 2, 1 for ECSM waveforms. Relu activation function is used for all Autoencoder layers, and sigmoid function is used in the output layer. Using sigmoid function adds a non-linearity learning capability to the Autoencoders that enables them to learn more complex data.

To calculate the compression ratio, the size of both encoded data and the encoder size need to be calculated to compare with the original data size on disk. Given S number of waveforms, P number of encoded parameters, B number of bytes to store the encoded parameters, then the encoded waveform size, E, if given by equation (3). On the other hand, given R number of actual data points per waveform saved on disk; R = 50,100,150 and 1000 data points for each ECSM waveform, then the uncompressed data size, O, is given by equation (4). Considering that the total size needed to store the encoded data on disk is the encoded waveform size plus the decoder size, D, needed to decode the waveforms. Hence, the effective compression ratio is calculated by equation (5).

$$E\,(Encoded\,Waveform\,Size) = (S * P * B) \tag{3}$$

$$O\,(Original\,Waveform\,Size) = (S * R * B) \tag{4}$$

$$Effective\,Compression\,Ratio = O/(E + D) \tag{5}$$

4.3.2. Training Autoencoder models

All Autoencoder models with all possible values of the parameter P (16, 8, 4, 2, 1) were trained on the normalized WF data. The used computer configuration is: Intel Core I7, 16 GB RAM, with 6 GB NVIDIA GeForce RTX 2060 GPU. Keras library v2.4 with GPU-enabled Tensorflow backend is used to model Autoencoder models. Adam optimization is used in the training process and mean squared error, MSE, is used as a loss function. The training process was designed to abort training if the loss value deteriorates 50 iterations in sequence. Single Autoencoder model is trained on a combined input of NOR, NAND, INV rising or falling-edge waveforms.

4.3.3. Voltage waveform data compression results

To evaluate the performance of the trained Autoencoders, all the normalized waveforms were compressed using Python implementation of gzip, bz2 techniques [7], and SVD. Autoencoders of 2 parameters gave between 39× and 45× better compression ratios than gzip and bz2 techniques at less degree of accuracy. For SVD, all the normalized waveforms are packed in a matrix A, of size 71,400 times the number of sampling points). The matrix A is analyzed using SVD. Different ranks of the sigma matrix are selected to produce the compressed version of matrix A. The SVD data compression at different ranks are evaluated and compared to our proposed Autoencoder-based compression at different number of encoding parameters. Table 6 shows that Autoencoders of 2 parameters were able to produce 1.67× better compression ratio results than SVD technique of rank 16, and less than 0.66% average

Table 6
Best Autoencoding 1000-points ECSM Waveform Results Against bz2, gzip and SVD techniques.

	Falling-Edge WF					Rising-Edge WF				
	bz2	gzip	SVD Rank = 16	Autoencoder, Model ID = 2		bz2	gzip	SVD Rank = 16	Autoencoder, Model ID = 2	
				2 param	4 param				2 param	4 param
% error	0	0	Avg/Max	Avg/Max	Avg/Max	0	0	Avg/Max	Avg/Max	Avg/Max
@ 0.8VDD			0.7/11.8	0.6/66	0.34/3.2			2.3/26.4	0.6/11	0.26/5.9
@ 0.5VDD			0.4/7.6	0.5/5.6	0.24/2.1			0.6/10.6	0.42/7	0.21/2.3
@ 0.2VDD			1.0/17.5	0.6/35	0.27/3.6			1.6/26.6	0.6/27	0.27/4.9
Mean corrcoef	1	1	0.99957	0.9995	0.99988	1	1	0.99954	0.9994	0.99985
Loss (MSE)	0	0	1.7E-05	5.2E-05	1.27E-05	0	0	2.09E-05	5.8E-05	1.37E-05
Disk size-MB	188.5	161.6	6.9	4.15	4.15	188.5	161.6	6.9	4.15	0.26/5.9
Compression Ratio	2.29	2.67	62.5	104.06	86.14	2.36	2.71	62.5	104.06	86.14
Compression time (s)	49.6 s	32 s	0.974 s	4.6 s	4.6 s	49.6 s	32 s	0.974 s	4.6 s	4.6 s
Decompression time (s)	16.2 s	10 s	0.212 s	2.6 s	2.6 s	16.2 s	10 s	0.212 s	2.6 s	2.6 s

percentage error compared with SPICE simulation. The offline training time is 600 s for 100 epochs it can be enhanced with better hardware resources and smaller encoding models.

The same experiment is repeated with 150-points sampled waveforms at variable time range. Table 7 shows that Autoencoders of 2 parameters also give 1.79× better compression ratio results than the nearest SVD model of rank 4, and less than 0.45% average percentage error compared with SPICE simulation. Autoencoders of 2 parameters also gave between 40× and 55× better compression ratios than gzip and bz2 techniques at less degree of accuracy. This accuracy is achieved with offline training time of 100 s for 100 epochs. The encoded 150-points sampled waveforms remain overall better than the encoded 1000-points sampled waveforms in average and maximum percentage errors. Fig. 4 shows reconstructed falling and rising edge waveform example.

Autoencoding either 1000 and 150 points waveforms with 4 parameters gives smaller errors rates than the reported SVD rank 16 and rank 4 error rates yet they are still greater than SVD rank 32, and rank 8 error rates respectively. The smaller error rates with 4 parameters encoding come at the cost of reduced compression ratios, but it remains a tradeoff option. However, encoding waveforms with larger parameters doesn't significantly reduce the error rates.

Tables 6 & 7 show that the best model ID is different for different number of waveform data points, but the best model ID is the same for the same number of waveform data points. Model ID#2 structure will used to compress different 1000 points waveforms without exploring other model structures, similarly model ID#17 structure will be used to model different 150 points waveforms.

4.4. Deep learning Waveform-Delay Model, DL-WFDM

Deep learning waveform delay model, DL-WFDM, is proposed instead NLDM LUT or DL-NLDM models. In DL-WFDM, autoencoded input/output waveforms parameters are proposed to model cell delays instead of the cell delay and the transition time used in the NLDM. Fig. 5 shows our proposed new cell delay training methodology. First, each input waveform is encoded as well as its associated output waveform using the corresponding rising or falling edge Autoencoder encoder model. Since sigmoid activation function is used for the middle Autoencoder layer i.e., the encoder output layer, there is no need to normalize the input waveform parameters or the expected output waveform parameters.

The encoded input waveform parameters and the associated effective capacitance values are applied as input data to train the deep learning network. The deep-learning network is trained on these encoded input waveform parameters in addition to the associated effective capacitance input value to produce the expected encoded output waveform parameters.

Table 8 shows that the proposed DL-WFDM model results. Using 1000-points sampled encoded waveforms to train the new model resulted in up to 1.14% average percentage error, and up to 18.13% maximum percentage errors, for key waveform points compared with SPICE simulation. Using 150-points sampled encoded waveforms, DL-WFDM model gives up to 1.33% average percentage error, and up to 20.57% maximum percentage errors. Comparing the delay time errors in Tables 2, 3. 4 with the 50% VDD crossings in Tables 8 & 9 shows that DL-WFDM gives slightly more average percentage errors, but better maximum percentage

Table 7
Best Autoencoding 150-points ECSM Waveform Results Against bz2, gzip and SVD techniques.

	Falling-Edge WF					Rising-Edge WF				
	bz2	gzip	SVD Rank = 4	Autoencoder, Model ID = 17		bz2	gzip	SVD Rank = 4	Autoencoder, Model ID = 17	
				2 param	4 param				2 param	4 param
% error	0	0	Avg/Max	Avg/Max	Avg/Max	0	0	Avg/Max	Avg/Max	Avg/Max
@ 0.8VDD			0.6/16.4	0.34/5.5	0.18/2.8			0.42/5.9	0.32/5.32	0.13/2.25
@ 0.5VDD			0.4/3.3	0.24/5.0	0.11/1.4			0.48/4.4	0.23/2.71	0.11/1.29
@ 0.2VDD			0.5/15.3	0.43/9.0	0.12/1.6			0.48/6.3	0.34/5.34	0.23/3.81
Mean corrcoef	1	1	0.99632	0.99914	0.99975	1	1	0.99316	0.998632	0.99973
Loss (MSE)	0	0	1.78E-04	5.1E-05	9.31E-06	0	0	2.15E-04	7.27E-05	1.05E-05
Disk size-KB	6805	5501	245	136	136	7593	6048	245	136	136
Compression Ratio	1.35	1.67	37.5	67.45	35.35	1.21	1.51	37.5	66.85	35.35
Compression time (s)	16 s	2.4 s	0.06 s	0.62 s	0.62 s	7.1 s	1.2 s	0.06 s	0.62 s	0.62 s
Decompression time (s)	1.45 s	0.24 s	0.03 s	0.52 s	0.52 s	1.45 s	0.24 s	0.03 s	0.52 s	0.52 s

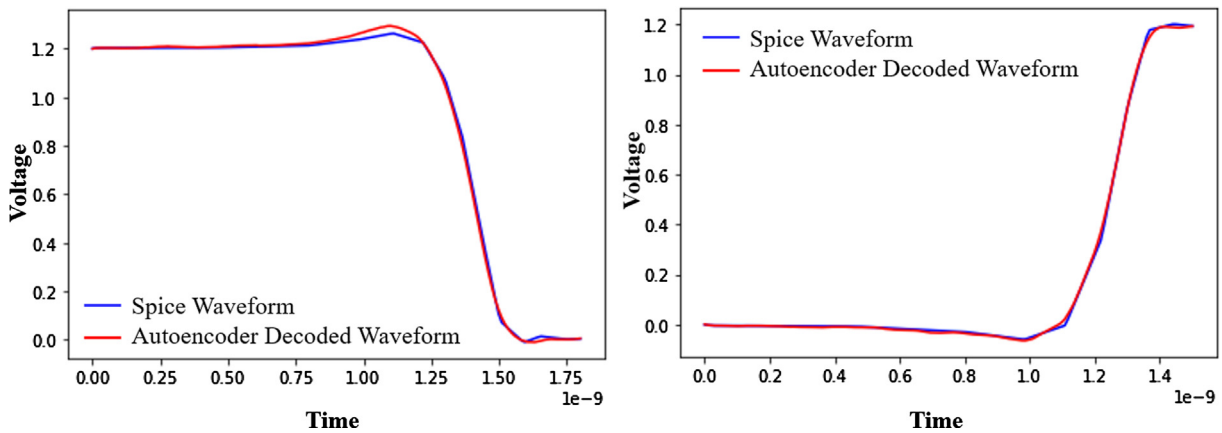

Fig. 4. Falling and rising decoded waveforms against Spice waveforms.

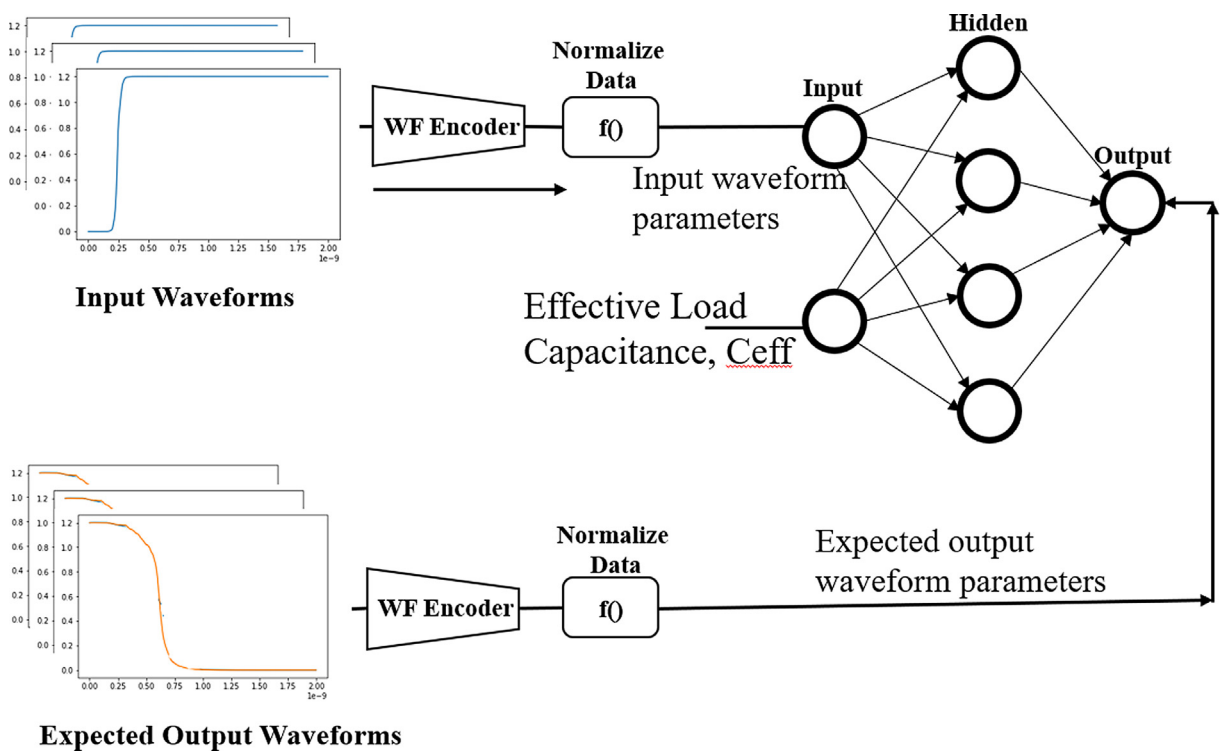

Fig. 5. Training DL Cell model with encoded input/output waveform parameter.

Table 8
DL-WFDM Model Training Results using 1000-points Sampled Waveforms.

	Falling Edge Waveform Output, Model ID = 4						Rising Edge Waveform Output, Model ID = 4					
Cell	**NOR**		**INV**		**NAND**		**NOR**		**INV**		**NAND**	
% error	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max
@ 0.8VDD	0.61	5.77	0.87	16.38	1.06	14.90	0.59	4.89	1.02	18.13	0.57	7.78
@ 0.5VDD	0.42	5.44	0.72	13.17	0.93	4.95	0.38	2.45	0.54	10.08	0.35	5.46
@ 0.2VDD	0.62	3.48	1.14	17.3	1.21	7.41	0.41	4.20	0.58	4.61	0.41	4.39
Avg Corrcoef	0.9995		0.9986		0.9985		0.96435		0.9636		0.9643	
MSE	4.8E-05		1.14E-04		1.41E-04		6.99E-05		1.4E-04		6.72E-05	
Disk size-KB	38						38					
Time - (s)	Training time = 40 s – Retrieving time = 0.4 s						Training time = 40 s – Retrieving time = 0.4 s					

errors for falling and rising edges for most of cell types. Though DL-WFDM using 1000-points compressed waveforms are better than those using 150-points compressed waveforms, the latter is recommended because of the shorter training time, and retrieving time of the DL-WFDM and smaller compression, decompression, and offline training time of the related Autoencoders.

Table 9
DL-WFDM Model Training Results using 150-points Sampled Waveforms.

	Falling Edge Waveform Output, Model ID = 3						Rising Edge Waveform Output, Model ID = 4					
Cell	**NOR**		**INV**		**NAND**		**NOR**		**INV**		**NAND**	
% error	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max
@ 0.8VDD	0.82	10.63	1.33	13.94	0.99	9.5	0.87	5.15	0.94	19.31	0.88	9.04
@ 0.5VDD	0.49	6.62	1.05	9.98	0.79	7.93	1.02	6.14	0.94	18.96	0.83	10.08
@ 0.2VDD	0.55	6.41	0.91	8.47	0.66	6.14	1.00	6.06	1.05	20.57	0.93	10.28
Avg Corrcoef	0.9962		0.9886		0.9976		0.9943		0.9893		0.9932	
MSE	2.21E-4		5.04E-04		1.69E-04		2.42E-04		4.57E-04		2.9E-04	
Disk size-KB	15						38					
Time - (s)	Training time = 20 s – Retrieving time = 0.11 s						Training time = 20 s – Retrieving time = 0.1 s					

5. Conclusions and future work

In this research paper, DL-NLDM model is proposed, designed, and trained on input and output waveform delay and transition time in addition to capacitive load values. Two cell-delay models are trained on rising and falling edge waveform transitions. Average delay percentage error values of less than 1.4% is achieved compared to SPICE simulation outperforming average percentage errors of the standard 7×7 LUT size and outperformed the non-standard 100×100 LUT in maximum percentage errors. Waveform compression allows storing more complex waveforms data to raise the accuracy of the current source models without increasing the technology file size or degrading the performance. In this research, deep learning non-linear Autoencoders are used to compress voltage–time waveforms used in effective current source model. The performance of several deep learning Autoencoder models at different number of encoding parameters is evaluated against the SVD compression technique. Compression ratio of 104 at up to 0.66% average percentage error is achieved after encoding 1000-points sampled voltage–time waveforms. This compression ratios are 1.67× better than the nearest rank SVD results and 39 to 45x better than gzip and bz2 compression techniques. Better compression ratios are also achieved at less accuracy figures with other Autoencoder models and number of parameters. Autoencoding 150-points sampled waveforms with 2 parameters gives 1.79× better compression ratio than the nearest rank SVD compression with up to 0.45% average percentage error compared with SPICE simulation. Those Autoencoders gave 40 to 55x better than gzip and bz2 compression techniques. Autoencoders require large time to be trained, and larger compression and decompression time compared to the SVD algorithm, however training and compression are offline operations that can be accepted for the sake of better compression and accuracy. The decompression time can be enhanced with better hardware resources and smaller encoding models.

Future work will focus on enhancing both training time and run time of the proposed DL-NLDM and DL-WFDM models is key to adopt these models in practical digital design flows. Future work also includes enhancements to Autoencoders models and training algorithms to achieve higher compression ratios in less offline training time and less decoding time with lower error rates.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Kahng AB, Lienig J, Markov IL, Hu J. In: VLSI physical design: from graph partitioning to timing closure. Dordrecht: Springer Netherlands; 2011. p. 219–64.
[2] Ramalingam A, Singh AK, Nassif SR, Orshansky M, Pan DZ. Accurate waveform modeling using singular value decomposition with applications to timing analysis. In: 2007 44th ACM/IEEE design automation conference. p. 148–53.
[3] Berz D., Engstler M., Heindl M., Waibel F. Comparison of lossless data compression methods Technical reports in computing science No. CS-07-2015. University of Applied Sciences Kemptem.
[4] Goodfellow I., Bengio Y. and Courville A., "Deep Learning". MIT Press, http://www.deeplearningbook.org, pp. 164-223 and pp. 499-523, 2016.
[5] Li L, Yu W. Efficient and accuracy-ensured waveform compression for transient circuit simulation. IEEE Trans Comput Aided Des Integr Circuits Syst 2021;40(7):1437–49. doi: https://doi.org/10.1109/TCAD.2020.3020496.
[6] Abrishami MS, Pedram M, Nazarian S. CSM-NN: current source model based logic circuit simulation - a neural network approach", IEEE 37th international conference on computer design (ICCD). Abu Dhabi, United Arab Emirates 2019;2019:393–400.
[7] Python data compression library, https://docs.python.org/3/library/archiving.html.
[8] Sharma R., "Characterization and modeling of digital circuits", ISBN-10: 1983144827, pp.75-120, paripath.inc, 2016.
[9] Hatami S, Feldmann P, Abbaspour S, Pedram M. Efficient compression and handling of current source model library waveforms. In: 2009 Design, automation & test in Europe conference & exhibition. p. 1178–83. doi: https://doi.org/10.1109/DATE.2009.5090841.
[10] Saurabh S, Mittal P. A Practical methodology to compress technology libraries using recursive polynomial representation. In: 2018 31st international conference on VLSI Design and 2018 17th international conference on embedded systems (VLSID). p. 301–6. doi: https://doi.org/10.1109/VLSID.2018.80.
[11] Silicon Integration Initiaitive, https://si2.org/os-downloads.
[12] Zhang S, Zhai J, Chen J, He Q. Autoencoder and its various variants. IEEE international conference on systems, man, and cybernetics, 2018.
[13] Synopsys Inc., CCS Timing Technical White Paper version 2.0.
[14] Fournier Q, Aloise D. Empirical comparison between autoencoders and traditional dimensionality reduction methods. In: 2019 IEEE second international conference on artificial intelligence and knowledge engineering (AIKE). p. 211–4. doi: https://doi.org/10.1109/AIKE.2019.00044.
[15] Raslan W, Ismail Y. Deep learning autoencoder-based compression for current source model waveforms. 28th IEEE International conference on electronics, circuits, and systems. ICECS; 2021. doi: https://doi.org/10.1109/ICECS53924.2021.9665573.
[16] Wang Y, Yao H, Zhao S. Auto-encoder based dimensionality reduction. Neurocomputing 2016;184:232–42.